Neural Computing and Applications (2025) 37:8181–8192
https://doi.org/10.1007/s00521-022-08114-3

S.I.: DATA PROCESSING TECHNIQUES AND APPLICATIONS FOR CYBER-PHYSICAL SYSTEMS (DPTA-2021)

Performance optimization of computing task scheduling based on the Hadoop big data platform

Yang Li[1] · Xinhong Hei[1]

Received: 27 July 2022 / Accepted: 22 November 2022 / Published online: 25 December 2022
© The Author(s) 2022

Abstract
Hadoop, a distributed computing framework that can efficiently process large-scale datasets, has been used by an increasing number of organizations as the basic computing framework to build cloud computing platforms. Improving its execution efficiency is a hot research direction in the industry, and the scheduling problem is a key factor affecting the execution efficiency of Hadoop. It is very important to identify its shortcomings and improve them. This paper examines and analyses the optimization of computing task scheduling performance based on the Hadoop big data platform. This paper first analyses Hadoop big data processing. Hadoop has high scalability. Computing nodes can be added at any time, and they can participate in cluster work through simple configuration. The paper discusses the improvement in the Hadoop resource scheduling algorithm. The task scheduling algorithm in the Hadoop-based data task localization proposed in this paper is compared with the default algorithm used in the Hadoop task scheduling algorithm. The former shows better local data in all four jobs, there are more data localization tasks, and the expected goal is achieved. The effectiveness of the algorithm is verified, and the performance is improved by 30%.

Keywords Task scheduling · Hadoop big data · Particle swarm algorithm · Genetic algorithm

1 Introduction

The largest advantage of the Hadoop platform is that developers do not need to understand its internal implementation details, and its underlying automatic encapsulation completes complex parallelization techniques. In this case, technicians do not need to write complex cloud computing program codes; they only need to call the corresponding interface. While Hadoop has many advantages, there is still much room for improvement because Hadoop's development time is not long enough. In actual use, the related implementation details in the built-in MapReduce are not very good due to the short development time. When Hadoop is designing task scheduling, it implicitly assumes no difference in computing power between nodes. In reality, heterogeneous clusters are very common. According to the distribution of computing power, the ability to do more work can improve not only resource utilization but also platform execution efficiency. Therefore, it is very useful to study and improve the data locality and the difference in computing power. This paper thus examines and analyses the optimization of computing task scheduling performance based on the Hadoop big data platform. It is expected to make a certain contribution to optimizing task scheduling performance.

This paper examines and analyses the optimization of computing task scheduling performance based on the Hadoop big data platform. Compared with the default task scheduling algorithm used by Hadoop, the task scheduling algorithm for intratask data localization based on Hadoop proposed in this paper has a better data localization effect in the running process of four jobs. There are more data localization tasks, the expected goal is achieved, the

✉ Xinhong Hei
heixinhong@xaut.edu.cn

Yang Li
liyang_walker@stu.xaut.edu.cn

[1] College of Computer Science and Engineering, Xi'an University of Technology, Xi'an 710048, Shaanxi, China

effectiveness of the algorithm is verified, and the performance is improved by 30%. The reason is that during task scheduling, we compute the data localization saturation for each node. Low-saturation nodes take priority so that high-saturation nodes do not preempt tasks corresponding to data blocks stored by low-saturation nodes.

The innovation of this paper is reflected in the following: (1) This paper analyses and discusses Hadoop big data processing and simultaneously studies the improvement in the Hadoop resource scheduling algorithm. (2) This paper conducts experimental research on the performance optimization of computing task scheduling within Hadoop jobs and analyses the results.

2 Related work

According to the research progress abroad, different researchers have conducted corresponding cooperative research in task scheduling. Lu P conducted a comparative study of DoT scheduling in a fixed and flexible grid ML-IDCON. He proposed a DoT scheduling algorithm that works well for both types of networks [1]. Zheng proposed a novel ensemble method that schedules task sets to assign each task's locked cache contents to local caches and second-level caches [2]. Ahmad introduced a comparative analysis of existing simulators and visualization techniques, which are helpful for feasibility analysis of real-time input tasks for IoT embedded applications [3]. Considering the influence of observation time on imaging quality, Li proposed an agile satellite task scheduling based on a constraint satisfaction model and discrete differential evolution combined with variable neighbourhood search [4]. However, these scholars' explorations of task scheduling lack certain technical demonstrations. We found that there are better studies on task scheduling based on Hadoop big data, and we checked the relevant literature on Hadoop big data.

Some scholars have also conducted research on Hadoop big data. The main aim of Agarwal was to describe the concepts of big data, cloud computing, big data as a service and Hadoop as a service on cloud platforms [5]. To improve the efficiency of HadoopCluster in big data collection and analysis, Dadheech proposed an algorithm system that can meet the needs of protected discriminative data in HadoopClusters and improve performance [6]. Based on the built Hadoop big data platform, Zheng determined parameters such as the state vector, distance matching principle, and neighbourhood value through numerical tests. A short-term traffic flow prediction method based on the Hadoop big data platform was constructed to realize the short-term prediction of road traffic flow [7]. However, these scholars did not examine and analyse the optimization of computing task scheduling performance based on the Hadoop big data platform but only discussed its significance unilaterally.

3 Hadoop big data platform

3.1 Hadoop big data processing

Hadoop is the basic structure of a distributed system developed by the Apache Foundation. Users can develop distributed software on their own without knowing the distributed infrastructure. The advantages of the cluster are given full play to realize fast computing and storage. Hadoop is the Java source code in Google's cloud computing system, which includes HDFS and MapReduce [8]. It has high reliability, high scalability, high efficiency, high fault tolerance, and low cost. Many scholars use Hadoop as their research platform when they are engaged in research in the field of cloud computing [9]. Hadoop can be widely used in the field of big data processing mainly due to its capacity for data extraction, transformation and loading (ETL). Hadoop's distributed architecture makes the big data processing engine as close to the memory as possible, which is more suitable for batch operations, such as ETL, because such batch results can be sent directly to storage. Figure 1 shows the scope of cloud computing services.

One of the most commonly used applications of Hadoop is web search. Although it is not only a software architecture but also a parallel data processing engine, its performance is very good. Hadoop can distribute the computing process to a large number of cheap servers in the cluster for execution, the cost is low, and the low service price makes it easy for users to adopt [10]. Unlike some computing frameworks that "move data to tasks", Hadoop adopts a "move programs to data" model. During the execution process, Hadoop replicates the program running the dependent library to all task nodes, which improves the cluster performance and further reduces the I/O load [11]. Once a failed node is detected, Hadoop will retry execution on a different node, so the presence of a single compute node will not cause cluster inconsistencies. In addition, Hadoop has high scalability, and computing nodes can be added at any time to participate in cluster work through a simple configuration [12]. Figure 2 shows the Hadoop cluster distribution environment. The core part of the Hadoop distributed computing platform includes HDFS, the processing flow of MapReduce, the Hive data warehouse tool Hive, and the distributed database Hbase.

HDFS realizes the distributed storage of data, which solves a large number of problems existing in many distributed file systems. It includes the amount of data that can be stored (TB level or petabyte level), the reliability of

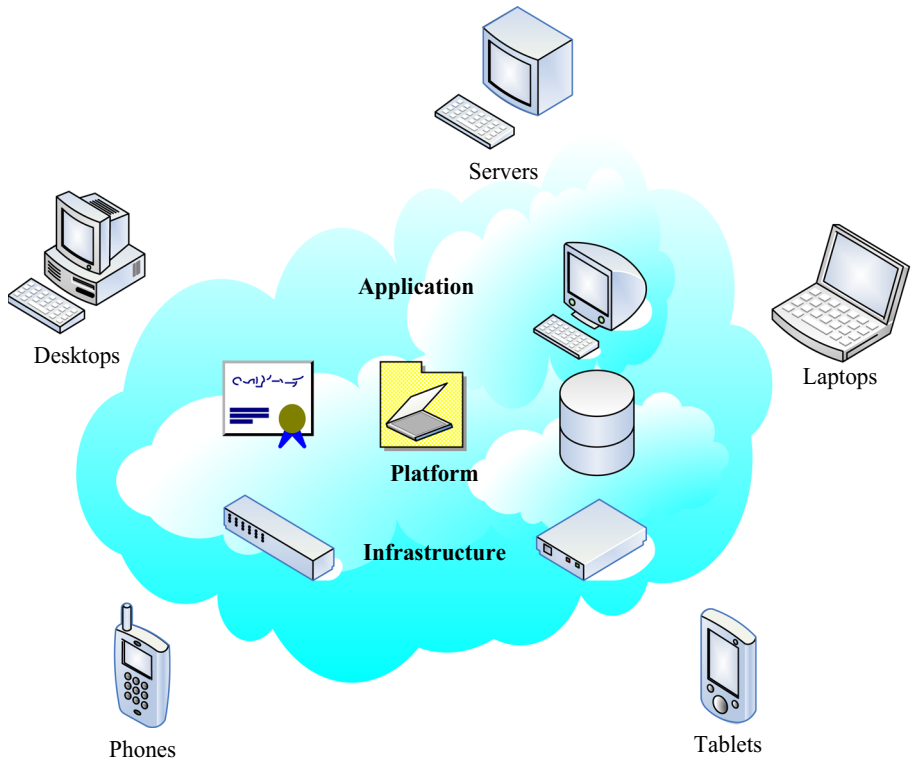

Fig. 1 Scope of cloud computing services

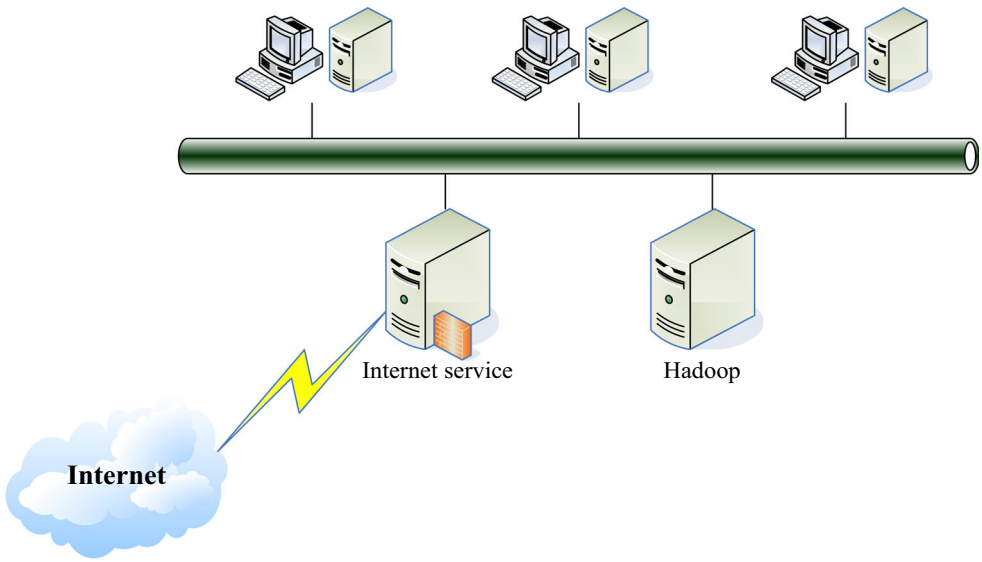

Fig. 2 Hadoop cluster

storage, and better integration with Hadoop's Map Reduce framework [13].

As shown in Fig. 3, HDFS is a typical master/slave structure, and the roles of master and slave are played by name node and data node, respectively [14]. Name nodes are responsible for managing the HDFS file system, including namespaces and file blocks. When saving files, HDFS divides a single file into several blocks according to a fixed size and can copy the blocks by setting the replication factor. These blocks are randomly stored on different data nodes, which greatly increases the storage capacity and availability of HDFS [15]. Each data node periodically sends heartbeats and block reports to the named node to check the status of the data node and whether the file data on the node is consistent with that recorded by the named node [16].

Because the named node has a single failure problem, when the named node fails, the backup named node can assume the job of the named node [17].

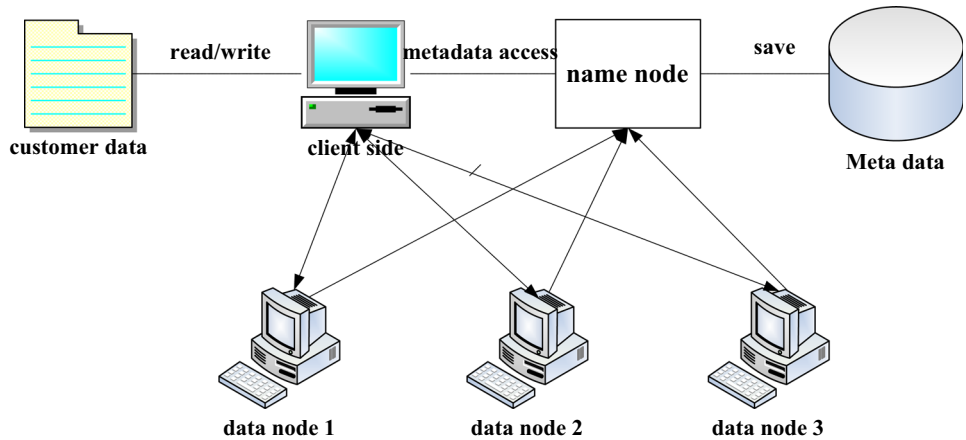

Fig. 3 HDFS architecture

3.2 Improvement of the Hadoop resource scheduling algorithm

In the Hadoop platform, cluster resources are uniformly managed and allocated by the resource scheduler. It is a pluggable component in Resource Manager. To facilitate expansion, the system predefines some specifications for users to implement resource scheduling policies by themselves [18].

Aiming at problems of low resource utilization and unbalanced load in cloud computing environments, a new method combining a hybrid genetic algorithm and particle swarm algorithm is proposed. It is applied to the scheduling of cloud computing resources to improve resource utilization [19, 20].

(1) Traditional optimization algorithm

Intelligent optimization algorithms include mainly genetic algorithms, simulated annealing algorithms, differential evolution algorithms, and particle swarm algorithms. Intelligent optimization algorithms are usually designed for specific problems, such as job scheduling, machine learning, and control systems [21]. Next, we introduce the particle swarm algorithm and genetic algorithm in detail.

Particle swarm optimization (PSO) is an evolutionary algorithm whose main goal is to graph the irregular behaviour of birds to realize intelligent simulation of the foraging process [22–24]. As shown in Fig. 4, PSO simulates the foraging behaviour of birds. A flock of birds haphazardly searches for food, and in this area, they only find a piece of food. Not a single bird knew there was food. However, they are well aware of the distance from their current location to the food. Therefore, the best way is to find food. The easiest and most effective method is to search close to the prey. Foraging behaviour is an instinctive animal behaviour. When animals are hungry and thirsty, they will naturally look for food and water to meet their physiological needs so that they can survive. Animals that lose their ability to find food will die. The particle swarm algorithm compares the movement area of birds to the solution space for solving problems. The individual guides the movement of the entire population with the information of adjacent individuals and, after many iterations, shows the possibility of obtaining the optimal solution.

The method takes the group as the background and transfers the individuals in the group to a better position by calculating the adaptability of the surrounding environment. The following formulas are updated formulas for particle velocity and particle position:

$$u_{nt}^{h} = r * u_{nt}^{h-1} + s_1 * e_1 * \left(ibest_{nt} - a_{nt}^{h-1}\right) + s_2 * e_2 * \left(ibest_t - a_{nt}^{h-1}\right) \quad (1)$$

$$a_{nt}^{h} = a_{nt}^{h-1} + u_{nt}^{h-1} \quad (2)$$

where $t = 1, 2, 3, \ldots$, and T represents the size of the search space and u_{nt}^{h} represents the t-dimensional component of the flight velocity vector of the h-th iteration particle n. a_{nt}^{h} represents the t-th dimension component of the h-th iteration particle n's position vector, and r is the customary weight that adjusts the search range of the solution space.

In the iterative method, particles have certain maximum and minimum velocities in the search space, and the goal is to set an interval to prevent particles from leaving the solution space. In general, both the maximum speed $V_{\max}$ and the minimum speed $V_{\min}$ are set as constants defined by the user. When the update is complete, the maximum and minimum velocities of the particles are updated. The update formula is as follows:

$$V_{nt} = V_{\min}, \quad \text{if } V_{nt} < V_{\min} \quad (3)$$

$$V_{nt} = V_{\max}, \quad \text{if } V_{nt} > V_{\max} \quad (4)$$

In the iterative process of particle swarm optimization, the combination of inertia, self-cognition and social

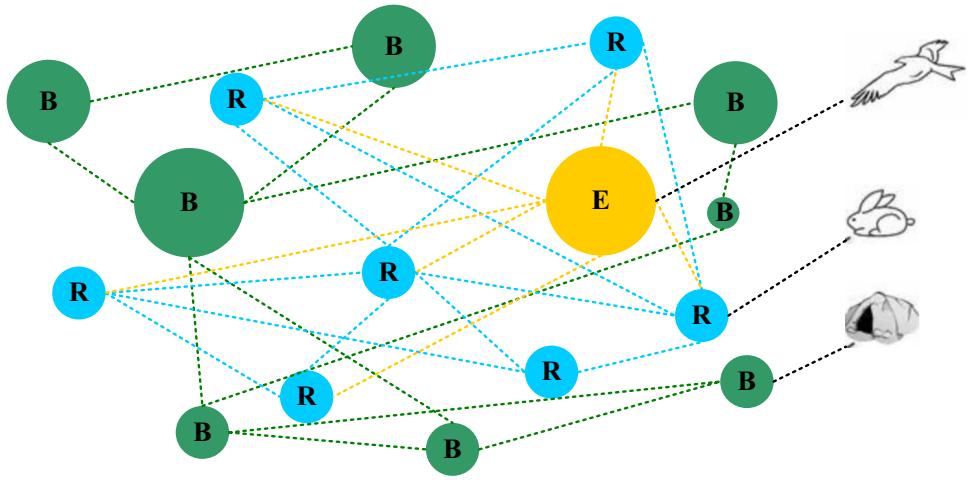

Fig. 4 Particle swarm optimization

experience allows the population to have a good convergence speed in the solution space.

The genetic algorithm is an adaptive global optimal probabilistic search method based on genetic evolution theory. Similar to natural evolution, the genetic algorithm follows an extremely strict survival law of the fittest through continuous evolution to find more suitable populations, and the best choice is optimal. In the process of evolution, individuals evolve according to a certain adaptive standard and towards the optimal direction for the individual. Then, the optimal solution is generated through corresponding coding rules, and a more reasonable solution can be obtained. The process of searching for the global optimal solution by the genetic algorithm is iterative until the termination condition of the algorithm is met. When a genetic algorithm is used to solve the problem, every possible solution is coded into a "chromosome", that is, an individual, where several individuals constitute a population.

(2) Cloud computing resource scheduling objective function

Resource scheduling refers to a method in which each resource user configures resources according to certain usage principles in a specific resource environment. The MapReduce programming framework involves concepts such as jobs, resources, storage devices, and scheduling processes. The content introduced next is to model the computing method of cloud computing programming and finally obtain the objective function of resource scheduling. This metric is measured by cluster execution time. This function can be applied to the genetic algorithm as a fitness function to make the calculation process more scientific. There are generally two ways to schedule resources for computing tasks: adjusting the use of resources allocated to a computer or offloading computing work to another computer. For edge computing resource allocation and task scheduling problems, an accurate method is proposed based on the analysis of system characteristics, but the algorithm complexity is high and consumes more computing resources, which is not suitable for large-scale problems. Most work uses heuristic strategies to allocate resources and schedule tasks. Although faced with large-scale tasks and isomerized resources, such methods are easy to design, easy to implement, and do not occupy too many computing resources, but it is difficult to achieve good overall optimization results. Intelligent algorithms are applicable to complex problems with strong constraints and multiple objectives and have strong scalability, but they are difficult to apply to scenarios with high real-time requirements, such as distributed online problems.

The scheduling problem of cloud resources includes three levels: scheduling resources, scheduling virtual resources (such as virtual machines), scheduling physical resources, and scheduling physical resources. In cloud computing, tasks and resources are not in one-to-one correspondence but are first associated with tasks and resources. The resources are then mapped to the corresponding physical devices to implement job scheduling. In short, the following 5 tuples can describe a resource scheduling process:

$$C = \{V, U, B, Q_{vu}, Q_{ub}\} \tag{5}$$

In this process, the task Q_{vu} of the system is assigned to the computer centre, and Q_{ub} is assigned to the corresponding storage device. Therefore, the key to resource planning is determining how to arrange resources for the storage device. The assignment matrix of task V to physical device B is denoted as:

$$\mathrm{ETD}_{qp} = \mathrm{ETD}(v_n Q_{vu}, b_h) \quad 1 \le n \le q, 1 \le h \le p \tag{6}$$

The earliest completion time of task v_n on the physical device b_h through the mapping relationship Q_{vu} is recorded as:

$$\text{Finish}(v_n Q_{vu}, b_h) = \text{Start}(b_h) + \text{ETD}(v_n Q_{vu}, b_h) \tag{7}$$

The total time it takes to assign tasks on physical device b_h can be expressed as:

$$\text{Sum}(b_h) = \sum_{n=1}^{q} s_{nh}\text{Finish}(v_n Q_{vu}, b_h) \tag{8}$$

$$s_{nh} = \begin{cases} 1, & v_n Q_{vu} = b_h \\ 0, & v_n Q_{vu} \neq b_h \end{cases} \tag{9}$$

Therefore, the total time for all tasks to execute through the resource scheduler is:

$$\text{total}(V) = \sum_{h=1}^{p} \text{Sum}(b_h) \tag{10}$$

The resource scheduling algorithm makes the running time of the task the shortest, which is the minimum value in the above formula.

$$\text{Goal}(V) = \min \sum_{h=1}^{p} \text{Sum}(b_h) \tag{11}$$

(3) Improved genetic algorithm

The traditional genetic algorithm is a typical optimization method. Based on the genetic algorithm, this paper optimizes the resource allocation characteristics in cloud computing to make it converge more quickly. On this basis, the fitness function is introduced to solve the shortcomings of the traditional genetic algorithm.

The standard genetic algorithm uses binary encoding, which includes a character set {0,1}, and these genes form binary-encoded strings. The problem addressed in this paper is how to classify tasks into a continuous task during the sorting process. If binary encoding is used, a continuous function becomes a discrete function. The largest drawback of binary encoding is that the Hamming distance is large, it is difficult to pass crossover and mutation, and the precision is not high. Real coding uses Euclidean distance to shift the genetic algorithm closer to the problem space, so we use real coding.

The genetic algorithm simulates the principle of best adaptation in nature for the search process. In this process, the fitness function is the criterion for the quality of individuals in the population. In cloud computing, task completion time is an important indicator of the merits of cloud resource allocation algorithms. The greater the algorithm's degree of adaptation is, the better its performance is, which can be expressed as follows:

$$g = 1 \Big/ \min \sum_{h=1}^{p} \text{Sum}(b_h) \tag{12}$$

A dynamic penalty function is introduced into the fitness function, and a smaller fitness value is proposed for those who are not satisfied. Its penalty function can be expressed as:

$$i = (sv)^{\beta} \sum_{h=1}^{p} j_h^{\theta} \tag{13}$$

In the formula, v represents the number of evolutions; generally, $s = 0.5$, $\beta = \theta = 2$. Therefore, the fitness function can be combined with the dynamic penalty function as:

$$G = \frac{1}{\min \sum_{h=1}^{p} \text{Sum}(b_h)} + (sv)^{\beta} \sum_{h=1}^{p} j_h^{\theta} \tag{14}$$

Before the crossover operation, individual matching is generally performed by random matching. Assuming that there are Q individuals in a population, $Q/2$ pairs are randomly formed, and then the two individuals in this random combination are paired.

Since the real number coding method is adopted, the arithmetic crossover method can be used to perform the crossover operation on A_n and A_m to obtain the descendants of $A_n^{'}$ and $A_m^{'}$, respectively. The crossover operation can be expressed as follows:

$$A_n^{'} = \beta A_n + (1 - \varphi) A_m \tag{15}$$

$$A_m^{'} = \beta A_m + (1 - \varphi) A_n \tag{16}$$

Assume that the mutant individual is $Q = [q_1, q_2, \ldots, q_h, \ldots q_K]^v$, and the probability of mutating is I_q.

$$q_{h+1} = \begin{cases} q_h + \Delta(v, q_{h\max} - q_h), & \beta = 0 \\ q_h + \Delta(v, q_h - q_{h\min}), & \beta = 1 \end{cases} \tag{17}$$

$\Delta(v, a)$ represents a nonuniformly distributed random number in the interval [0, a], and its expression can be described as follows:

$$\Delta(v, a) = a\left(1 - \theta^{(1-K/V)^d}\right) \tag{18}$$

In the formula, d represents the parameter that determines nonuniformity, and θ is a random number in the interval [0,1].

The population obtained by the evolution of the genetic algorithm is used as the initial population, and the optimal resource allocation scheme is obtained by using the global optimization of the algorithm and the local optimization of the PSO. Figure 5 shows the resource scheduling process of cloud computing.

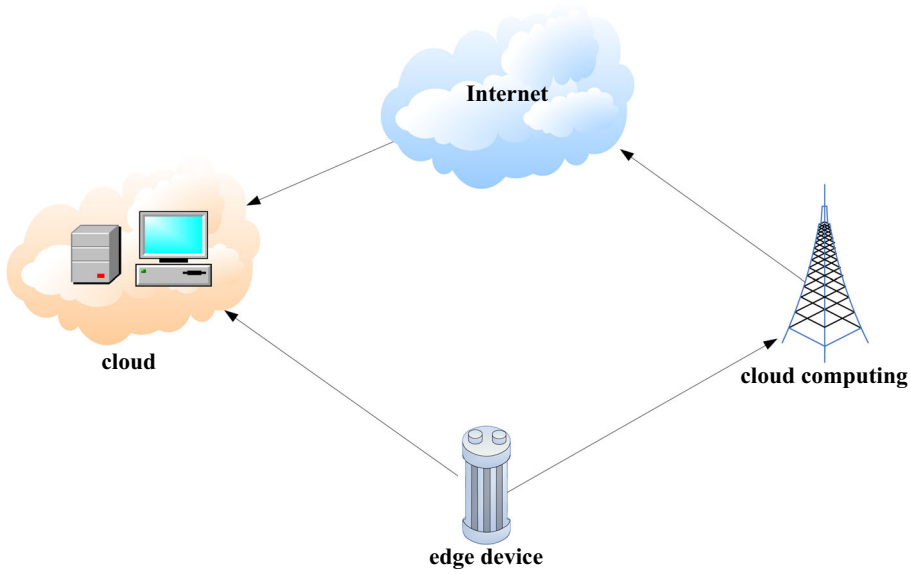

Fig. 5 Cloud computing resource scheduling

4 Experimental results of computing task scheduling performance optimization

4.1 Cluster configuration

The Hadoop cluster in this experiment consists of 6 PCs, one of which serves as the master node JobTracker. In addition, JobTracker serves as the name node and secondary name node, and the remaining machines serve as TaskTracker. It also acts as a data node. Each PC is connected through 100 M internet. In addition, the hardware configuration of the PC, such as memory size and CPU performance, is not the same, that is, the cluster is heterogeneous. The operating system used in this cluster is Ubuntu 14.04, and JDK version 1.7 is uniformly installed on the operating system. The version number of Hadoop is 1.2.0.

An overview of the composition of this cluster is given in Table 1. The hardware configuration of the master node is given in Table 2. Since each slave node is heterogeneous, the specific configuration differs, so the specific configuration parameter table of the slave node is not given.

Table 1 Hadoop cluster composition overview table

Node name	Node role
Node 1	JobTracker NameNode SecondaryNameNode
Node 2	TaskTracker DataNode
Node 3	TaskTracker DataNode
Node 4	TaskTracker DataNode
Node 5	TaskTracker DataNode
Node 6	TaskTracker DataNode

Table 2 Master node hardware configuration

Main parameter	Specific configuration
Number of CPU cores	4
CPU GHZ	2.5 GHZ
Memory size	8 GB
Hard disk size	1 T

4.2 Experimental data

This study uses the word count program WordCount that comes with Hadoop as a test program to evaluate the performance of the scheduler. The program counts the number of occurrences of each word in one or more files. We can download a batch of log files through the internet. From small to large, the data in this experiment are 512 M, 1 G, 1.5 G, and 2 G. Table 3 shows the experimental data situation.

This experiment focuses on the proportion of data localization computing tasks within the job and the job execution time when the same size job file uses different task scheduling algorithms in the Hadoop cluster. These results can be viewed through the monitoring webpage provided by Hadoop. The parameter Data-localmaptasks provided by the webpage represents the number of data localization tasks in the map task of the modified job, and the ratio can be calculated according to the number of map tasks. The task completion time can be obtained by subtracting the task start time from the completion time.

The specific experimental process is as follows: Step 1: Configure the Hadoop cluster environment and perform

Table 3 Experimental data situation

Job number	Job size	Number of data blocks	Number of Map tasks	Reduce tasks
Job 1	512 M	9	9	1
Job 2	1 G	18	18	1
Job 3	1.5 G	27	27	1
Job 4	2 G	36	36	1

different configurations according to the task scheduler used. Step 2: Upload the file to be calculated to HDFS. Step 3: Under different task schedulers, submit log files of the same size for calculation in the same way. Step 4: View the running results of each job under different task schedulers through the results provided by the webpage and calculate statistics. Step 5: Compare the experimental results, that is, the data locality and execution time of the job, to draw conclusions about the experiment. We use the current Hadoop default task scheduling algorithm and the data localization task scheduling algorithm based on Hadoop jobs proposed in this paper to conduct the experiments.

4.3 Experimental results

(1) Data locality analysis

The data locality of a job is an important indicator of the efficiency of job execution on a Hadoop cluster, which is reflected by the proportion of data localization tasks in a job. A data localization task means that when a computing node executes a task, the required data blocks are stored on the current node. The calculation process does not need to be transmitted from other nodes through the network; it can be directly read and locally calculated. The larger the proportion of data localization tasks is, the fewer the tasks that need to be transmitted through the network in the entire job, which reduces both the average execution time of all tasks and the job execution time. Statistics about tasks executed by jobs in Hadoop can be viewed through the web page. The proportion of data localization tasks in a job can be obtained based on the number of data localization computing tasks and the total number of computing tasks.

The line graph in Fig. 6 shows the trend of changes in the proportion of the number of data localization tasks in the job as the file of the calculated job increases. The job file to be calculated at the beginning is relatively small, and the number of divided tasks is also small, so each node needs only to process tasks corresponding to one or two data blocks. Therefore, the probability of nondata localization tasks is low, and a 100% data localization ratio is prone to occur. As the number of jobs to be calculated increases, the number of tasks increases. The proportion of data localization tasks generally does not reach 100%, and it can only keep approaching this value. The figure also shows that with an increase in the calculated job files and in the number of Map tasks, the number of data localization tasks in the job generally shows an increasing trend. This trend illustrates that large files have more data localization tasks than small files. The reason is that the Hadoop cluster is sharded according to a fixed size, so large files will be divided into more data shards than small files, and these data shards will be distributed and stored on each data node. The more nodes that are stored, the more convenient it is for nodes to allocate corresponding local work, which also indicates that Hadoop clusters can better handle large documents.

The bar chart in Fig. 7 shows a comparison of the proportion of data localization tasks in jobs with different task scheduling algorithms under the same size file. We can clearly find that the task scheduling algorithm based on data localization in Hadoop jobs proposed in this paper shows better data locality than the default task scheduling algorithm used by Hadoop during the four jobs. With more data localization tasks, the expected goal is achieved, the effectiveness of the algorithm is verified, and its performance is improved by 30%.

(2) Analysis of job execution time

The above experimental results show that the improved algorithm given in this paper shows obvious improvement in the data locality of the job. Next, we will analyse and compare the two algorithms in terms of job execution time, which is an important indicator to measure the processing efficiency of Hadoop. Faster job execution time means that the cluster can process more data per unit time, which means that the throughput of the entire cluster system increases. Moreover, improving the job execution time is often the most intuitive concern for developers.

In this regard, the line graphs of the job execution time of the two task scheduling algorithms are given. The vertical coordinate represents the execution time in minutes.

As shown in Fig. 8, as the amount of data that needs to be calculated increases, the performance of the cluster will increasingly improve. The data in this experiment are only GB-level files, and the amount of data processed by mainstream commercial companies on the Hadoop platform is PB and EB. At this time, the performance improvement will be more obvious. At the same time,

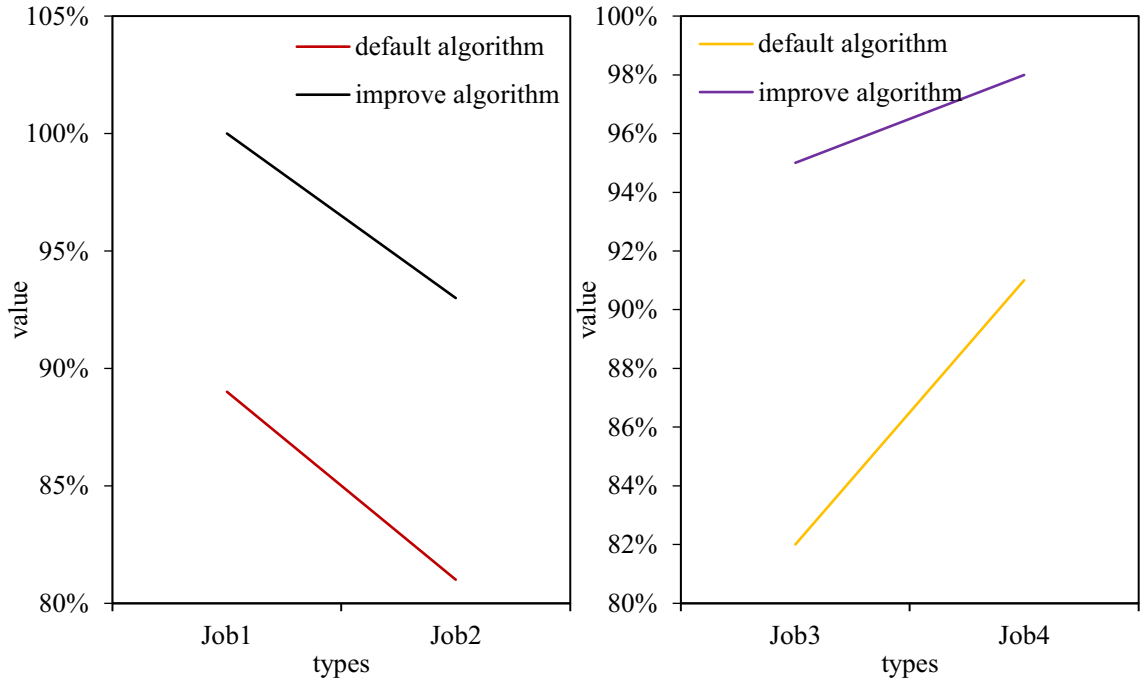

Fig. 6 Data localization comparison trend chart

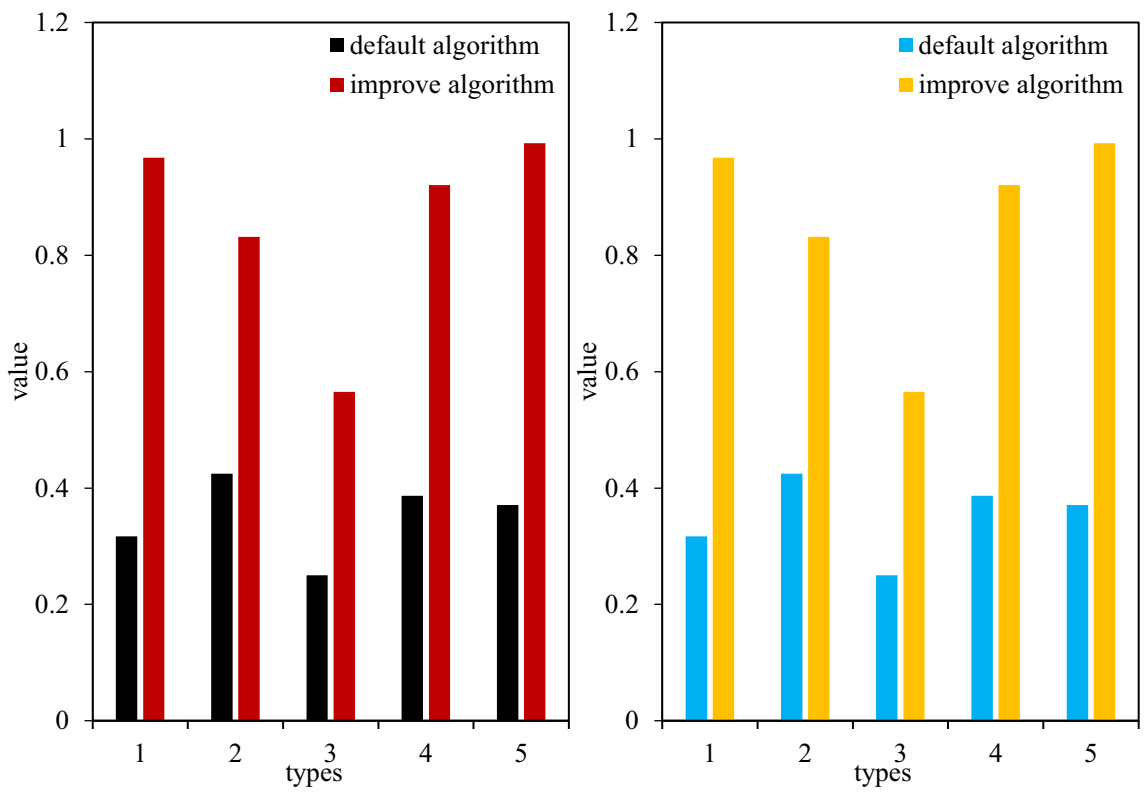

Fig. 7 Data localization comparison stage diagram

large-scale Hadoop clusters are generally deployed in a multirack structure in multiple data centres. Since this experiment cannot set up a cluster with multiple data centres in multiple places, the improvement in the algorithm proposed in this paper in reducing remote network transmission cannot be well reflected. In addition, as

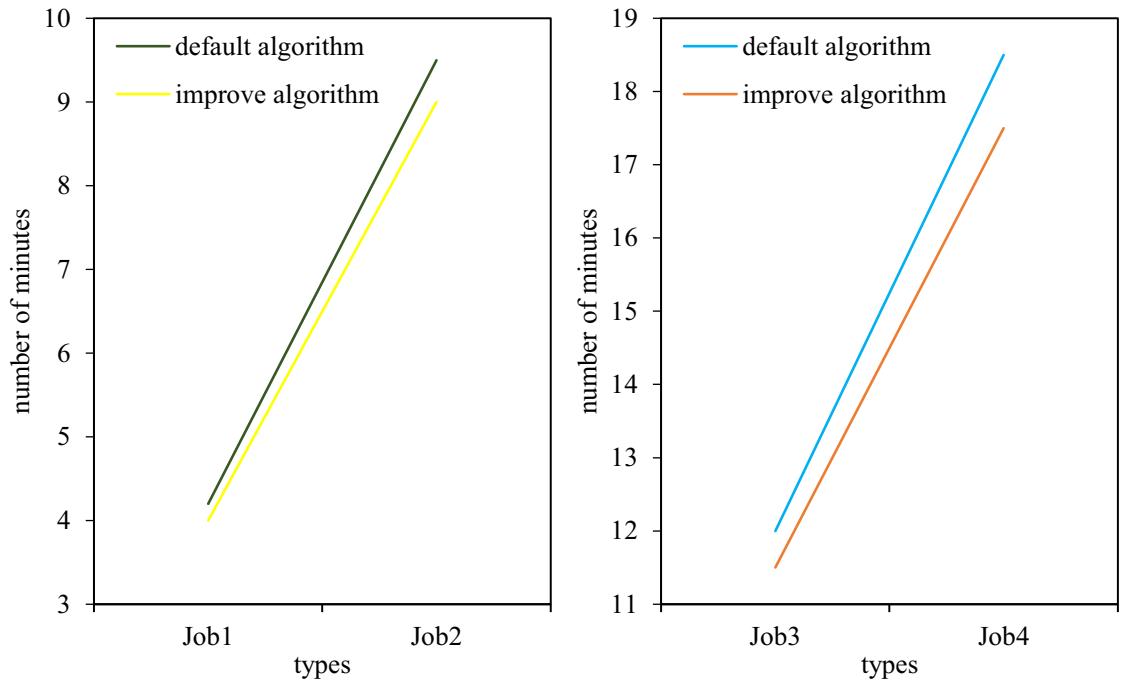

Fig. 8 Comparison trend of job execution time

mentioned in a previous article, the Hadoop platforms of large companies process hundreds of thousands of jobs per day. Therefore, any improvements that lead to increased cluster efficiency are significant.

Figure 9 shows that the execution time of each job is shortened under the task scheduling algorithm proposed in this paper, and the time reduction is more obvious with increasing data size. The reason is mainly the increased proportion of localized computing tasks as the amount of job data processed increases. The greater the total number of computing tasks is, the greater the number of data localization computing tasks is, and the reduction in the average execution time of each task leads to a reduction in the overall job execution time. Therefore, in the case of large-scale data computing, the improved algorithm proposed in this paper will perform better. At the same time, the experimental data show that the improved task scheduling algorithm obviously improves the data locality of the job and shortens the execution time of the job. It can thus improve the execution efficiency of the Hadoop platform and achieve the expected goal.

5 Conclusion

This paper uses six PCs to build a small heterogeneous Hadoop cluster as the experimental environment and prepares log files with sizes of 512 M, 1 G, 1.5 G, and 2 G. Then, it uses the WordCount program that comes with Hadoop as a test program. In the experiment, log files of the same size are calculated under different task scheduling algorithms, and the experimental results are counted. The data locality and execution time of the job are analysed in half. On the one hand, the improved algorithm proposed in this paper shows obvious improvement in data locality compared with the original task scheduling algorithm. The reason is that we calculate the data localization saturation of each node when scheduling tasks. Nodes with low saturation are executed first, and the tasks corresponding to the data blocks stored by nodes with low saturation will not be preempted by nodes with high saturation. In addition, the larger the log file used in the experiment is, the better the data locality is. The reason is the slicing mechanism of Hadoop. Larger files have more tasks, which also indicate that Hadoop clusters are suitable for computing large files. On the other hand, in terms of job execution time, the improved algorithm proposed in this paper has also significantly reduced. The reason is the increase in data localization computing tasks, as a result of which the network transmission overhead and the overall running time of the entire job are reduced. From these two aspects, the improved algorithm proposed in this paper achieves the expected goal, which proves the validity and reliability of the algorithm. In short, both cloud computing and Hadoop are in a period of rapid development, and there will definitely be many problems waiting for us to solve during

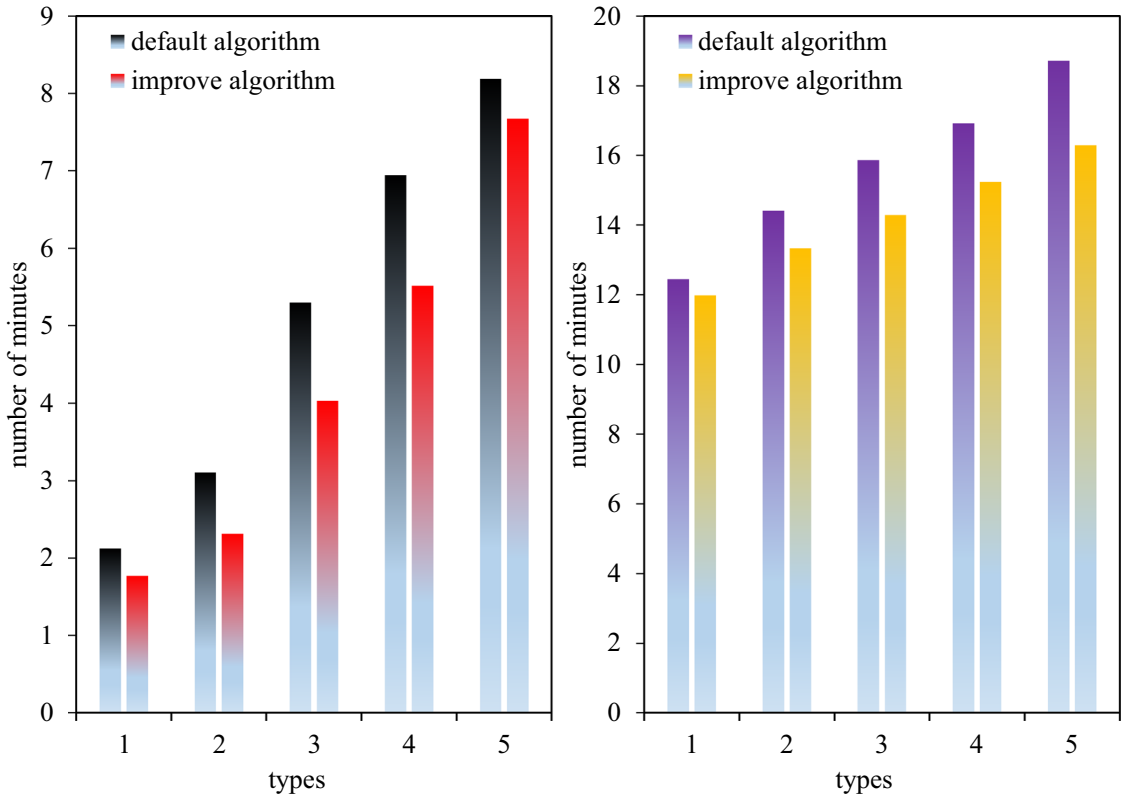

Fig. 9 Job execution time comparison stage diagram

development. We need to use the knowledge we have to generate new solutions to solve these problems and contribute to the development of the internet.

Funding National Natural Science Foundation of China (U20B2050, 62072368).

Data availability Data sharing is not applicable to this article, as no datasets were generated or analysed for the current study.

Declarations

Conflict of interest The authors declare that there are no conflicts of interest regarding the publication of this work.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Lu P, Zhu Z (2017) Data-oriented task scheduling in fixed- and flexible-grid multilayer inter-DC optical networks: a comparison study. J Lightwave Technol 35(24):5335–5346
2. Zheng W, Wu H, Nie C (2017) Integrating task scheduling and cache locking for multicore real-time embedded systems. ACM Sigplan Not 52(4):71–80
3. Ahmad S, Malik S, Kim DH (2018) Comparative analysis of simulation tools with visualization based on realtime task scheduling algorithms for IoT embedded applications. Int J Grid Distrib Comput 11(2):1–10
4. Li ZL, Li XJ, Sun W (2017) Task scheduling model and algorithm for agile satellite considering imaging quality. J Astronaut 38(6):590–597
5. Agarwal U (2017) Cloud computing BDaaS and HDaaS (big data as a service and Hadoop as a service). Int J Comput Sci Eng 5(11):131–134
6. Dadheech P, Goyal D, Srivastava S (2017) Performance improvement of heterogeneous Hadoop clusters using MapReduce for big data. Int J Comput Eng 5(8):211–214
7. Zheng Y, Zhou Y (2018) Short-term traffic flow prediction method based on Hadoop big data platform. IPPTA Q J Indian Pulp Pap Tech Assoc 30(4):616–623
8. Kumar S, Singh M (2019) A novel clustering technique for efficient clustering of big data in Hadoop ecosystem. Big Data Min Anal 2(4):240–247
9. Alarabi L, Mokbel MF (2017) A demonstration of ST-Hadoop: a MapReduce framework for big spatio-temporal data. Proc VLDB Endow 10(12):1961–1964

10. Gattoju S, Nagalakshmi V (2021) An efficient approach for bigdata security based on Hadoop system using cryptographic techniques. Indian J Comput Sci Eng 12(4):1027–1037
11. Oo MN, Parvin S, Thein T (2018) Forensic investigation through data remnants on Hadoop big data storage system. Int J Comput Syst Sci Eng 33(3):203–217
12. Kapil G, Agrawal A, Khan RA (2020) Big data security challenges: Hadoop perspective. Int J Pure Appl Math 120(6):11767–11784
13. Ramya P, Sundar C (2020) SecDedoop: secure deduplication with access control of big data in the HDFS/Hadoop environment. Big Data 8(2):147–163
14. Erraissi A (2019) Hadoop storage big data layer: meta-modeling of key concepts and features. Int J Adv Trends Comput Sci Eng 8(3):646–653
15. Srilakshmi S, Rao CM (2019) Big data performance evaluation in Hadoop eco system. Int J Comput Sci Eng 7(5):1131–1135
16. Aniket B, Shravani J, Sudarshan D (2019) Analysis of big crime data using pig script with Hadoop. Int J Comput Appl 178(27):17–20
17. Rochd Y (2018) Performance improvement of PrePost algorithm based on Hadoop for big data. Int J Intell Eng Syst 11(5):226–235
18. Lee T, Yu ES, Park K (2019) Detection of abnormal ship operation using a big data platform based on Hadoop and spark. J Korean Soc Manuf Process Eng 18(6):82–90
19. Kim T-Y, Kim S-H, Ko H (2021) Design and implementation of BCI-based intelligent upper limb rehabilitation robot system. ACM Trans Internet Technol 21(3):1–17
20. Li X, Sun Y (2020) Stock intelligent investment strategy based on support vector machine parameter optimization algorithm. Neural Comput Appl 32:1765–1775. https://doi.org/10.1007/s00521-019-04566-2
21. Srinivasulu B, Mebrahtu A (2017) Concepts and technologies of big data management and Hadoop file system. Int J Emerg Trends Technol Comput Sci 44(2):80–88
22. Liu X, Wang W, Zhu G (2017) Research and analysis of big data based on Hadoop. Bol Tec/Tech Bull 55(4):382–386
23. Ilayaraja M (2020) Particle swarm optimization based multihop routing techniques in mobile ADHOC networks. Int J Wirel Ad Hoc Commun 1(1):47–56. https://doi.org/10.54216/IJWAC.010105
24. Salam MA (2021) Intelligent system for IoT botnet detection using SVM and PSO optimization. J Intell Syst Internet Things 3(2):68–84. https://doi.org/10.54216/JISIoT.030203

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.